

AiFinPay × AutoGPT

B2B Integration Guide — give your agents a wallet and let them pay for services per call.

Audience: AutoGPT engineering team

Networks: Polygon & Solana mainnet

Model: Non-custodial · no KYC · no API key

AiFinPay is Stripe for autonomous AI agents. One line of code — `agent.pay(url)` — lets an AutoGPT agent hold a wallet, pay for any service per call, and settle a real on-chain payment, then receive the gated response. This guide shows the three ways to integrate, leading with the one that fits AutoGPT best: **MCP**.

01 Why this fits AutoGPT

AutoGPT agents run unattended and call paid services — inference, search, vector DBs, APIs. Today each needs a human's API key and credit card. With AiFinPay, the **agent itself** carries a wallet and pays per call — exactly the “stop building workflows, start hiring agents” vision, with the missing piece (money) filled in.

Each agent gets its own address, a USDC balance, a monthly spend limit, and a transaction history of what it spent on which service. An embedded wallet panel drops straight onto this.

02 Option 1 — MCP (recommended · zero-code)

AutoGPT, Claude Desktop, Cursor, Windsurf, Continue and Cline all speak **Model Context Protocol**. Add one block and every agent gains five payment tools automatically — no SDK calls to wire by hand.

```
// claude_desktop_config.json / ~/.cursor/mcp.json
{
  "mcpServers": {
    "aifinpay": {
      "command": "npx",
      "args": ["@aifinpay/mcp"]
    }
  }
}
```

The model now has these tools:

Tool	What it does
<code>agent_address</code>	Returns the agent's wallet address (to fund / display)
<code>agent_quote</code>	Price-checks an x402-gated URL before paying
<code>payable_fetch</code>	Fetches a URL; auto-settles the 402 challenge on-chain

<code>pay_with_split</code>	Direct B2B payment with the on-chain split
<code>quote_split</code>	Previews the split (merchant / treasury / fee)

That's the whole integration. The agent can now autonomously pay any x402-gated API and get the gated response back.

03 Option 2 — Python / Node SDK (programmatic)

```
# Python
pip install aifinpay-agent --pre

# Node / TypeScript
npm install @aifinpay/agent@alpha
```

The “agent pays in a few lines” example — the core of the whole product:

```
from aifinpay import Agent

agent = Agent.new() # fresh wallet / keypair
print("Fund this address:", agent.address)

# Autonomously settles the 402 challenge on-chain, returns the response
resp = agent.pay(
    "https://bridge.aifinpay.company/io-net/chat/completions",
    body={"model": "meta-llama/Llama-3.3-70B-Instruct",
          "messages": [{"role": "user", "content": "Hello"}]},
)
print(resp.json()["choices"][0]["message"]["content"])
print("tx hash:", resp.headers.get("x-payment-receipt"))
```

Node is identical: `Agent.new()` → `agent.pay(url, { body })`. Persist `agent.secret_b58` for a durable identity across restarts (`Agent.from_secret(...)`), and read balance with `agent.balance(asset="USDC", chain="polygon")` — that's what powers the wallet panel (balance + spend limit + history).

04 Option 3 — Drop-in AutoGPT loop (already written)

The SDK already ships a **ready-made AutoGPT example**: a headless, self-funding agent that funds itself once, then buys inference + search calls in a perpetual loop until its balance hits a floor.

```
# github.com/AiFinPay/sdk → examples/autogpt/loop.py
pip install aifinpay-agent --pre openai
export OPENAI_API_KEY=sk-...
python loop.py # prints an address to fund, then runs unattended
```

This is the canonical pattern for any headless agent operating on a fixed budget — the AutoGPT team can run it as-is to see the full lifecycle.

05 Under the hood — settlement & events

agent → SDK / MCP → x402 challenge (HTTP 402) → agent signs + pays
→ on-chain atomic split → gated response returned to agent

- Settlement **is** the transaction — non-custodial, no intermediate hold.
- Every successful call emits one on-chain `Payment` event on the verified `B2BSplitter` contract (Polygon mainnet).
- The protocol fee (1%) is collected automatically by the splitter — no revenue-share bookkeeping, no contract to sign.

Webhooks (real-time events)

Partners subscribe a URL to events and receive HMAC-signed deliveries with automatic retries:

- Events: `payment.intent`, `payment.completed`, `seat.reserved`, `b2b.pay_with_split.invoice`
- `POST /api/webhooks` to subscribe; each delivery is signed (`x-aifinpay-signature: t=<ts>,v1=<hmac>`) — verify with the shared secret, timing-safe.

06 Becoming a paid service (the other side)

If AutoGPT wants to *sell* a service to agents (not just spend), they can stand up a paid bridge in ~30 minutes and earn per call — full steps in the SDK's `PARTNER_ONBOARDING.md`. They keep their API key and upstream relationship; AiFinPay handles the payment rail and agent identity.

07 Reference

Resource	Where
SDK + all examples	<code>github.com/AiFinPay/sdk</code>
Quickstart (60-second first paid call)	<code>QUICKSTART.md</code>
MCP client matrix	<code>MCP_CONFIG.md</code>
AutoGPT example	<code>examples/autogpt/</code>
Partner onboarding (sell a service)	<code>PARTNER_ONBOARDING.md</code>
Live site	<code>aifinpay.company</code>

Suggested next step with AutoGPT

- They add the MCP block (Option 1) — instant payment tools.
- They run `examples/autogpt/loop.py` to see a self-funding agent live.
- Follow-up call to design the embedded wallet UI (balance + spend limit + per-service history) on top of the MCP tools.